Universidad del Quindío

Facultad de Ingeniería · Ingeniería de Sistemas y Computación

Ingeniería de Software II

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

Modelo C4 y vista C4+1 (escenarios)

Sistema Comanda — Gestión de Pedidos y Caja

Cliente de referencia: Restaurante El Buen Sabor

Sebastián José Lesmes Muñoz · Código 1006775047

Kevin David Puerta Hidalgo

Presentado a: Juan Manuel Cardona

Versión 0.3 · Armenia, Quindío · 19 de octubre de 2026

Repositorio: https://github.com/SSeanjjo/gestion-restaurantes

Contenido

1. Introducción

1.1 Descripción del sistema y propósito

Comanda es un sistema web de gestión operativa para restaurantes pequeños y medianos. Digitaliza el flujo completo del servicio del Restaurante El Buen Sabor, que hoy opera con comandas en papel, calculadora y cuaderno de caja: el mesero registra el pedido en una tablet, la comanda llega en tiempo real a la pantalla de cocina, el inventario de ingredientes se descuenta automáticamente según la receta de cada plato, el mesero recibe un aviso cuando el plato está listo, el cajero cobra con efectivo, tarjeta o pago dividido y emite la factura electrónica validada por la DIAN, y el administrador concilia y cierra la caja del día.

El propósito del sistema es reducir los errores en pedidos y cobros, agilizar la comunicación sala–cocina, controlar el inventario en tiempo real y cumplir la obligación legal de facturación electrónica (necesidades §2.3 del Student Book, requisitos de negocio BR-01 a BR-08).

1.2 Objetivos del informe

- Describir la arquitectura de la versión 0.3 del sistema en los cuatro niveles del **modelo C4** (contexto, contenedores, componentes y código) y en la **vista C4+1** de escenarios mediante diagramas de secuencia de los casos de uso críticos.
- Justificar las decisiones de diseño clave y su relación con los requisitos funcionales y no funcionales del Plan de Requisitos V3.
- Documentar los patrones arquitectónicos y de diseño (GoF) aplicados, los principios SOLID y las refactorizaciones realizadas sobre el código del CU-01.
- Explicar la estrategia de calidad y pruebas, el despliegue, la configuración y las medidas de seguridad, para facilitar el mantenimiento y la evolución del sistema.

1.3 Alcance de la versión 0.3

Tabla 1. *Alcance funcional de la versión 0.3*

Caso de uso	Estado	Funcionalidades
CU-01 Gestionar Pedido	Completo	F-01 a F-13 (incluye F-08: cola de comandas con ACK y reenvío)
CU-02 Cobro y Cierre de Caja	Completo (pasarela y DIAN con adaptadores simulados)	F-14 a F-20
CU-04 Gestionar Inventario	Parcial	F-21: stock, alertas de nivel mínimo y entradas
CU-03, CU-05, CU-06	Próximas versiones	Gestión de menú, reservas y reportes por período

2. Contexto del sistema

2.1 Descripción del dominio del problema

El dominio es la operación de un restaurante de comida casera con servicio a la mesa. Intervienen cuatro roles internos — mesero, cocinero, cajero y administrador — y dos sistemas externos: la **pasarela de pago bancaria** (IS-01) y el **servicio de facturación electrónica de la DIAN** (IS-02). El cliente del restaurante es un actor externo que no usa el sistema directamente pero es el principal afectado por los errores actuales.

Las entidades centrales del dominio son la **Mesa** (disponible, ocupada, reservada), el **Pedido** con sus **detalles** y su ciclo de vida (Registrado → En preparación → Listo para servir → Entregado → Facturado, o Anulado), el **ProductoMenu** con su **receta** de **ingredientes**, la **Factura** con sus **pagos** y el **CierreCaja** diario. Las reglas de dominio RD-01 a RD-05 del Student Book (mesa ocupada, consistencia del pedido, flujo de estados, restricción de precios y facturación obligatoria) se validan en el backend.

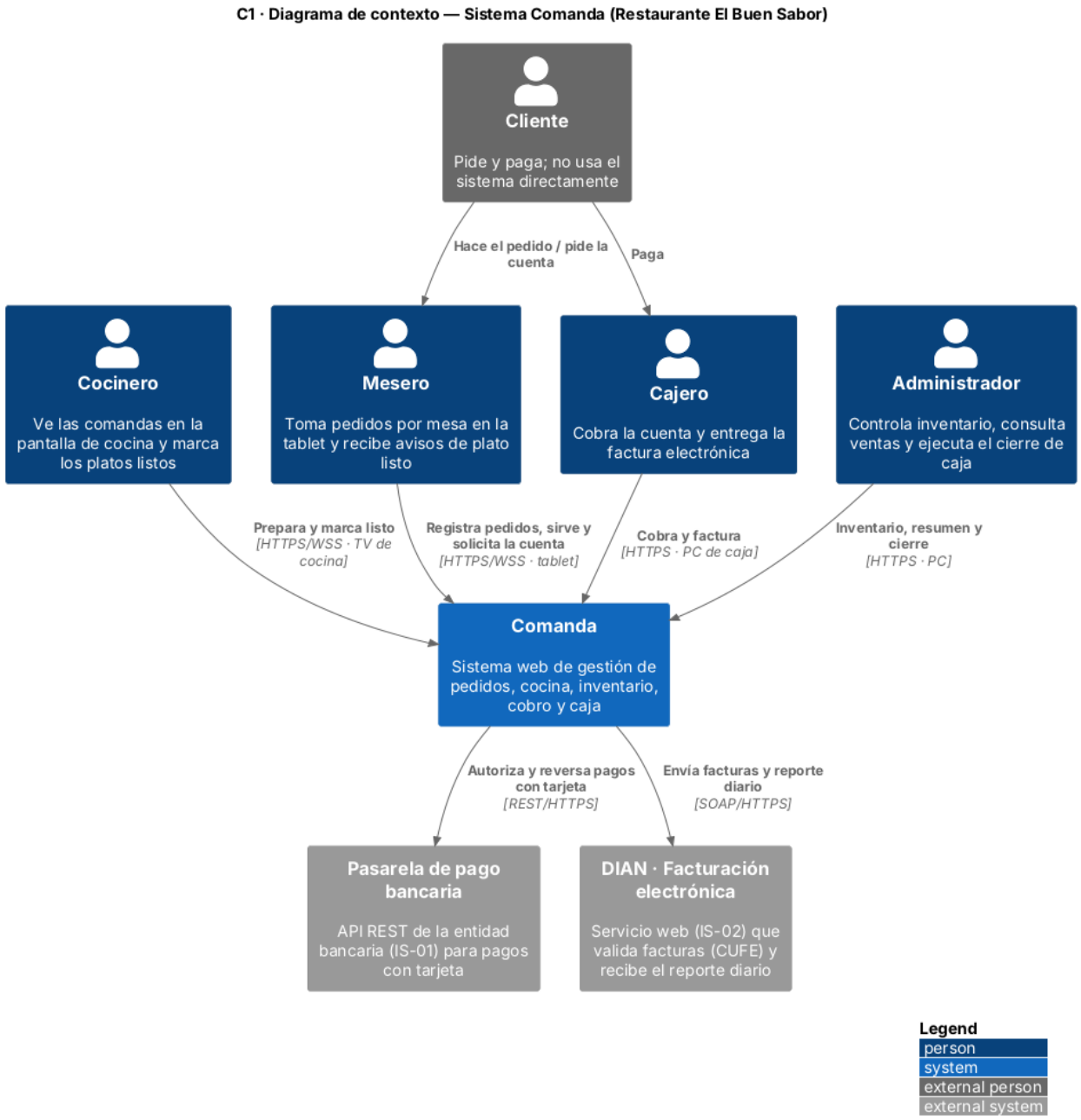


Figura 1. *C1 · Diagrama de contexto del sistema Comanda*

2.2 Requerimientos y restricciones del sistema

Requisitos funcionales

Los requisitos funcionales se especifican en el Plan de Requisitos V3 con la cadena de la norma ISO/IEC/IEEE 29148: requisito de negocio (BR) → de stakeholder (StR) → del sistema (SyR) → del software (SWR), cada uno con su criterio de verificación. La tabla resume las 21 funcionalidades.

Tabla 2. *Requisitos funcionales (resumen del Plan de Requisitos V3)*

ID	Funcionalidad	BR	Arquitectura
F-01	Seleccionar mesa y consultar su pedido activo	BR-01	
F-02	Consultar menú disponible por categoría	BR-01, BR-03	
F-03	Registrar pedido asociado a una mesa	BR-01	Afecta
F-04	Modificar pedido antes de confirmarlo	BR-01, BR-05	
F-05	Validar disponibilidad de ingredientes al confirmar	BR-03	Afecta
F-06	Descontar ingredientes del inventario	BR-03	Afecta
F-07	Enviar comanda a cocina en tiempo real	BR-02	Afecta
F-08	Almacenar y reenviar comandas ante pérdida de conexión	BR-04	Afecta
F-09	Visualizar comandas activas y tiempo en cola	BR-02	
F-10	Marcar plato / pedido como listo	BR-02	
F-11	Notificar al mesero que el plato está listo	BR-02, BR-05	Afecta
F-12	Marcar pedido como servido	BR-05	
F-13	Anular pedido	BR-01, BR-03	
F-14	Solicitar la cuenta de la mesa	BR-06, BR-05	
F-15	Consultar la cuenta con el total calculado	BR-06	
F-16	Registrar pago en efectivo con cálculo de cambio	BR-06	
F-17	Pago con tarjeta mediante la pasarela bancaria	BR-06	Afecta
F-18	Dividir el pago entre efectivo y tarjeta	BR-06, BR-05	
F-19	Generar factura electrónica y liberar la mesa	BR-07	Afecta
F-20	Conciliar y ejecutar el cierre de caja	BR-08	Afecta
F-21	Consultar inventario, alertas de mínimo y entradas	BR-03	

Requisitos no funcionales y atributos de calidad

Tabla 3. *Requisitos no funcionales*

ID	Atributo (ISO/IEC 25010)	Requisito	Cómo lo atiende la arquitectura
RNF-01	Seguridad	TLS hacia la DIAN y la pasarela	Salidas externas concentradas en adaptadores; TLS terminado en Nginx en producción
RNF-02	Eficiencia de desempeño	Comanda en cocina en ≤ 2,0 s	WebSocket/STOMP con publicación inmediata tras el commit
RNF-03	Usabilidad	Botones táctiles ≥ 44 × 44 px	Tokens de diseño compartidos en la SPA
RNF-04	Seguridad	RBAC; caja y cierre restringidos	JWT + @PreAuthorize por operación + permisos por tópico STOMP
RNF-05	Fiabilidad	Disponibilidad 99,5 % en horario	Contenedores con reinicio automático y healthchecks
RNF-06	Portabilidad	Responsive en Chrome y Safari	SPA Angular con CSS Grid/Flex
RNF-07	Mantenibilidad	Arquitectura limpia, API	Monolito modular, patrones GoF, Swagger,

ID	Atributo (ISO/IEC 25010)	Requisito	Cómo lo atiende la arquitectura
		documentada	pruebas y CI
RNF-08	Fiabilidad	No perder comandas (M-03)	Outbox con ACK, reenvío ordenado e idempotencia
RNF-09	Seguridad	Auditoría de operaciones sensibles	Usuario y fecha en anulaciones, movimientos, pagos, facturas y cierres
RNF-10	Fiabilidad	Consistencia transaccional	Transacciones, bloqueo pesimista y eventos AFTER_COMMIT

Restricciones de interfaz y de entorno

- **Hardware:** tablets Android/iOS (IH-01), TV o monitor de cocina (IH-02), PC de caja (IH-03), router Wi-Fi 802.11n o superior (IH-04) y datáfono integrado por la API del banco (IH-05).
- **Software:** navegadores Chrome y Safari recientes; base de datos relacional (IS-03); servidor web que expone HTTPS y WebSocket (IS-04).
- **Normativa:** facturación electrónica según la Resolución DIAN 000042 de 2020 y su Anexo Técnico (CUFE SHA-384); no almacenar el número completo de las tarjetas (buenas prácticas PCI DSS).
- **Despliegue:** servidor local o en la nube con Docker Compose; el sistema no requiere instalar software en los dispositivos de los usuarios.

2.3 Tecnologías utilizadas

Tabla 4. *Tecnologías utilizadas*

Capa	Tecnología	Uso en el sistema
Frontend	Angular 19 (standalone, signals), TypeScript 5.7	SPA con las cuatro interfaces IU-01 a IU-04
Tiempo real (cliente)	@stomp/stompjs 7	Suscripción a tópicos con reconexión automática
Servidor web	Nginx 1.27	Servir la SPA y proxy inverso de /api y /ws
Backend	Java 21, Spring Boot 3.4 (Web, Data JPA, Validation, Security, OAuth2 Resource Server, WebSocket, Actuator)	API REST, broker STOMP, reglas de negocio, tareas programadas
Persistencia	MariaDB 11.4, Hibernate, Flyway	Datos y migraciones versionadas (V1–V3 y semilla repetible)
Seguridad	JWT HS256 (Nimbus), BCrypt	Autenticación sin estado y contraseñas con hash
Documentación	springdoc-openapi (Swagger UI)	Documentación de la API REST (RNF-07)
Pruebas y calidad	JUnit 5, Mockito, AssertJ, JaCoCo, script de humo (curl + jq)	Pruebas unitarias, cobertura y prueba de extremo a extremo
Integración continua	GitHub Actions	Compilar, probar, construir imágenes y ejecutar la prueba de humo
Contenedores	Docker, Docker Compose v2	Empaquetado y despliegue reproducible
Modelado	PlantUML + C4-PlantUML, Jira	Diagramas C4/C4+1 versionados con el código; gestión del backlog

3. Arquitectura de alto nivel

3.1 Descripción general y patrones arquitectónicos

El sistema sigue una arquitectura **cliente–servidor** con una aplicación de una sola página (SPA) y un **monolito modular** en el backend. Dentro del monolito el código se organiza por **módulos de dominio** (mesa, menú, pedido, cocina, caja, inventario) y, dentro de cada módulo, por **capas** (controlador → servicio → repositorio). Los cambios relevantes se propagan mediante **eventos de dominio** (publicar/suscribir) que el backend difunde por WebSocket a las pantallas interesadas.

Tabla 5. *Patrones arquitectónicos utilizados*

Patrón arquitectónico	Aplicación en Comanda	Atributo de calidad favorecido
Arquitectura en capas	Controladores REST → servicios de aplicación → repositorios JPA → MariaDB	Mantenibilidad, analizabilidad
Monolito modular	Paquetes por dominio con dependencias explícitas; un solo despliegue	Simplicidad operativa; facilita extraer servicios a futuro
Cliente–servidor / SPA + API REST	Angular consume la API; Nginx sirve los archivos estáticos	Portabilidad, usabilidad
Publicar/suscribir (event-driven)	Eventos de dominio → PublicadorTiempoReal → tópicos STOMP	Desempeño (RNF-02), bajo acoplamiento
Puertos y adaptadores (hexagonal)	PasarelaPago y ServicioFacturacionElectronica con adaptadores intercambiables	Modificabilidad, testabilidad
Transactional Outbox	Tabla entrega_comanda escrita en la transacción del pedido	Fiabilidad (RNF-08)
MVC en el cliente	Componentes Angular (vista + controlador) y servicios (modelo)	Separación de responsabilidades

3.2 Diagrama de arquitectura de alto nivel

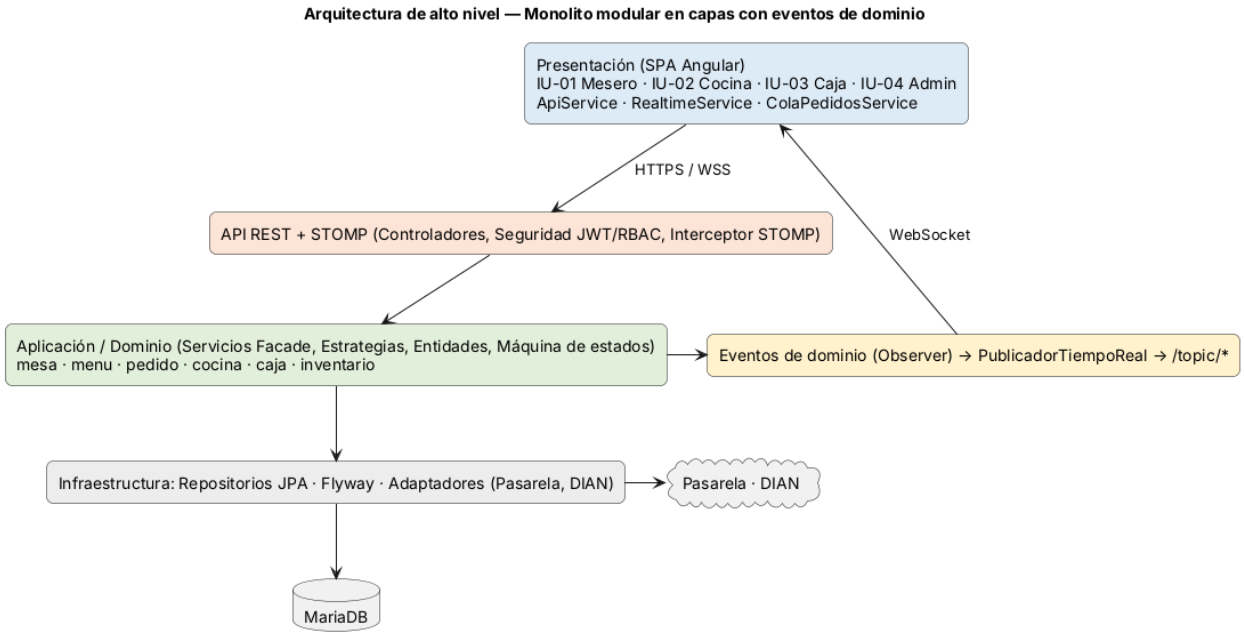


Figura 2. *Vista de alto nivel: capas y flujo de eventos*

El nivel C2 del modelo C4 muestra los contenedores desplegables: la SPA que corre en el navegador de cada dispositivo, Nginx como servidor web y proxy inverso, la API Spring Boot (que también actúa como broker STOMP en memoria) y la base de datos MariaDB, además de los dos sistemas externos.

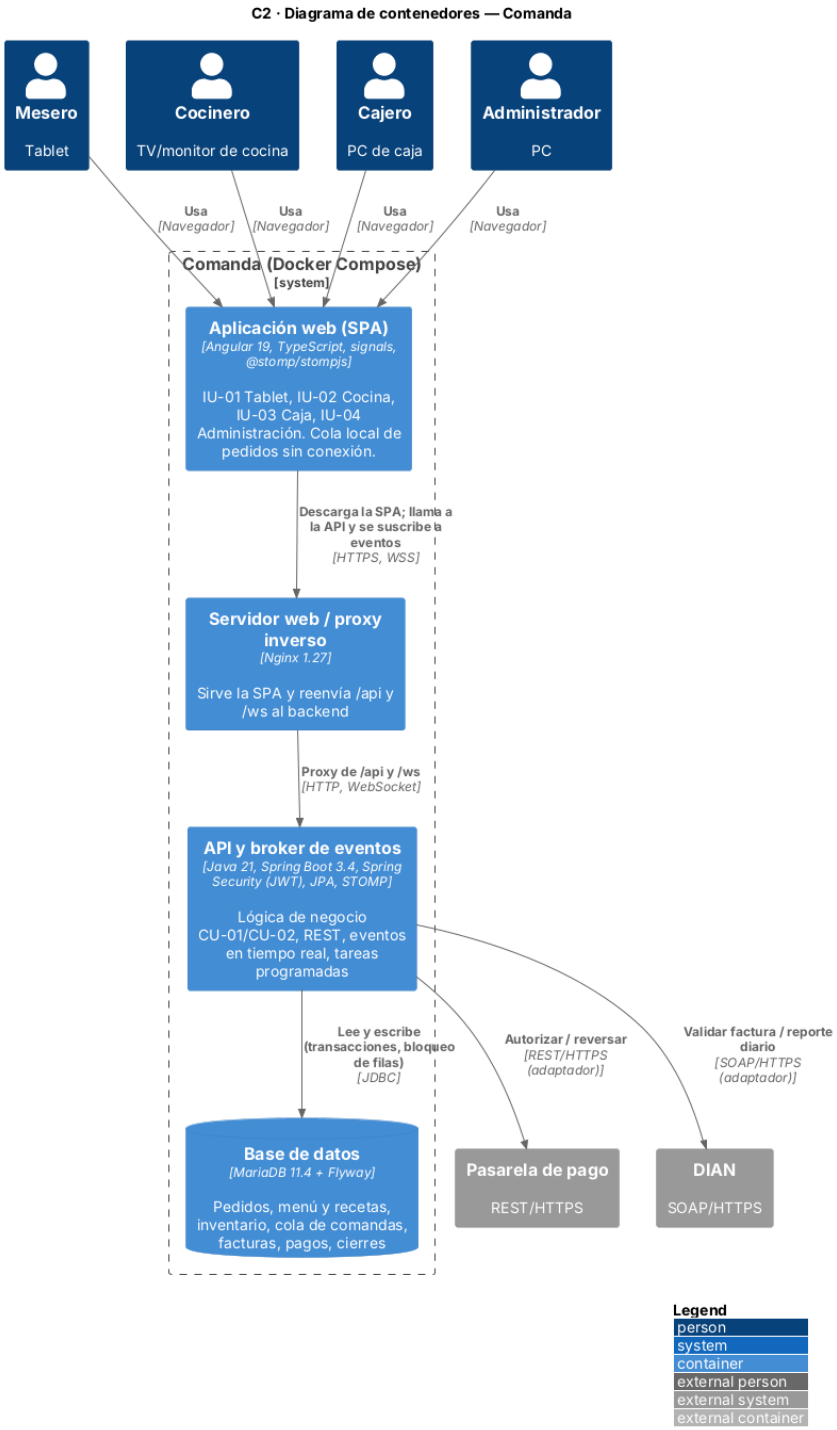


Figura 3. *C2 · Diagrama de contenedores*

4. Componentes y módulos

4.1 Componentes del backend y su responsabilidad

Tabla 6. *Componentes del backend*

Módulo / componente	Responsabilidad	Requisitos
auth · AuthController, TokenService, UsuarioSesion	Inicio de sesión, emisión de JWT y principal de la sesión WebSocket	RNF-04
config · SecurityConfig, WebSocketConfig, AutenticacionStompInterceptor, PoliticaSuscripcion	Seguridad REST y STOMP, CORS, broker de mensajes, OpenAPI	RNF-04, IS-04
mesa · MesaController, MesaService	Estado de las mesas y pedido activo de una mesa	F-01
menu · MenuService, ReglaDisponibilidad, DisponibilidadPorReceta, CalculadoraReceta	Menú con disponibilidad calculada por receta; catálogo de recetas	F-02
pedido · PedidoService, ConsumoIngredientesPedido, Pedido, EstadoPedido, RecordatorioPlatosListos	Registrar, preparar, notificar, servir, anular y solicitar cuenta	F-03 a F-14
cocina · ColaComandas, EntregaComanda, PresenciaCocina, EntregaComandaController	Cola persistente de comandas, ACK, reenvío y estado de conexión de cocina	F-08
caja · CajaService, CierreCajaService, Factura, Pago, Desglose, ResumenJornada	Cuentas, cobro, factura electrónica, conciliación y cierre	F-15 a F-20
caja.pago · ProcesadorPago, PagoEfectivo, PagoTarjeta, PasarelaPago	Estrategias de pago e integración con la pasarela	F-16 a F-18
caja.dian · ServicioFacturacionElectronica, DianSimulada, CalculadoraCufe	CUFE y validación de facturas; reporte diario	F-19, F-20
inventario · InventarioService, Ingrediente, MovimientoInventario	Stock, entradas y auditoría de movimientos	F-06, F-21
tiemporeal · Eventos, PublicadorTiempoReal	Eventos de dominio y su publicación STOMP después del commit	F-07, F-11
common · NegocioException, ManejadorErrores, JornadaOperativa	Errores con código estable y estado de la jornada (caja abierta/cerrada)	M-02

4.2 Componentes del frontend

Tabla 7. *Componentes del frontend*

Componente	Responsabilidad
MeseroComponent (IU-01)	Cuadrícula de mesas, menú por categoría, pedido en construcción, notificaciones, solicitar cuenta, anular; aviso de cocina desconectada y pedidos guardados en la tablet
CocinaComponent (IU-02)	Comandas por hora de llegada, temporizador de colores, marcar platos y "listo"; envía el ACK de recepción

Componente	Responsabilidad
CajaComponent (IU-03)	Mesas con cuenta, desglose, efectivo con cambio, tarjeta, pago dividido y vista previa de la última factura
AdminComponent (IU-04)	Métricas del día, inventario con alertas y entradas, conciliación, cierre e historial
ApiService	Una función por operación de la API REST
AuthService + authInterceptor + rolGuard	Sesión JWT, cabecera Authorization y redirección por rol
RealtimeService	Cliente STOMP autenticado; reconexión y resuscripción; aviso de reconexión para recargar el estado
ColaPedidosService	Cola local de pedidos confirmados sin red, con clave de idempotencia y reintento en orden

4.3 Relaciones entre componentes y módulos

Las dependencias entre módulos se mantienen en una sola dirección siempre que es posible. Donde un módulo de alto nivel necesitaba información de otro que a su vez depende de él, se introdujo una abstracción (principio de inversión de dependencias): por ejemplo, el módulo de pedidos pregunta si la caja está cerrada a través de la interfaz `JornadaOperativa`, que implementa `CierreCajaService`, y el módulo de cocina se entera de los pedidos nuevos escuchando el evento `PedidoCambiado`, sin que `PedidoService` lo conozca.

Tabla 8. *Dependencias entre módulos*

Módulo	Depende de	Mecanismo
pedido	mesa, menu, inventario, common (JornadaOperativa)	Inyección de dependencias; interfaz para la jornada
cocina	pedido (repositorio y DTO)	Escucha eventos de pedido (Observer)
caja	pedido, mesa, auth	Facade CajaService; estrategias y puertos propios
menu	inventario	Receta → Ingrediente
tiemporeal	DTO de mesa, pedido e inventario	Escucha todos los eventos de dominio
config	auth	Interceptor STOMP crea UsuarioSesion a partir del JWT

4.4 Diagramas de componentes (C3)

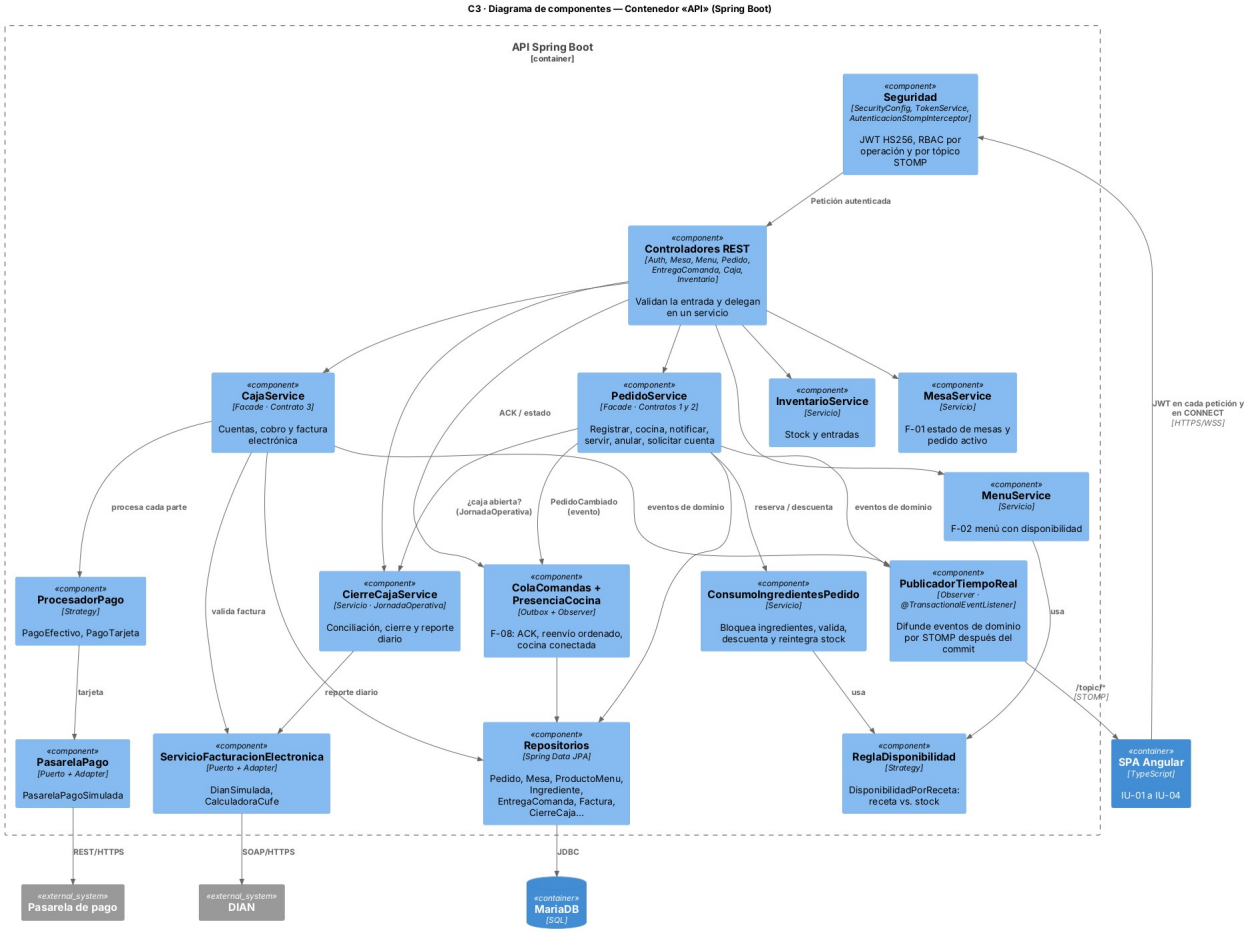


Figura 4. *C3 · Componentes del contenedor API (Spring Boot)*

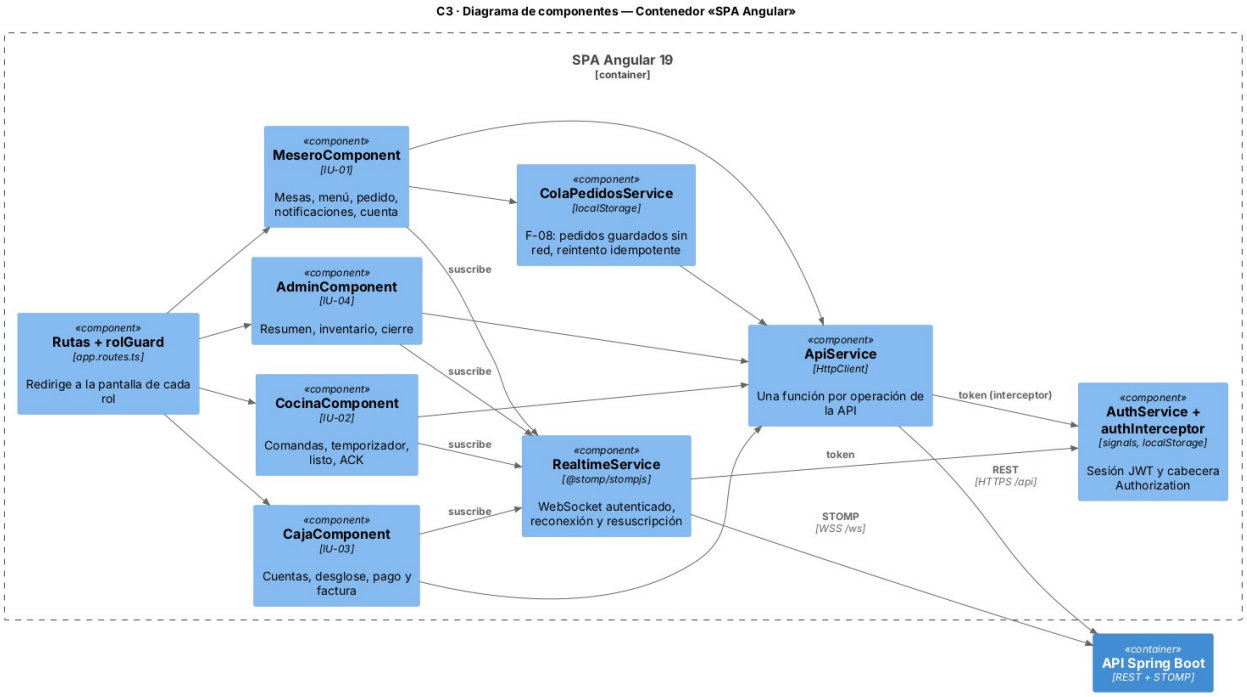


Figura 5. *C3 · Componentes del contenedor SPA (Angular)*

4.5 Nivel de código (C4)

El nivel C4 se documenta solo para las unidades críticas o complejas, como recomienda el modelo: el dominio del pedido con su máquina de estados, el módulo de caja (donde se concentran los patrones Strategy, Adapter y Builder) y la cola de comandas de F-08 junto con la seguridad del canal STOMP.

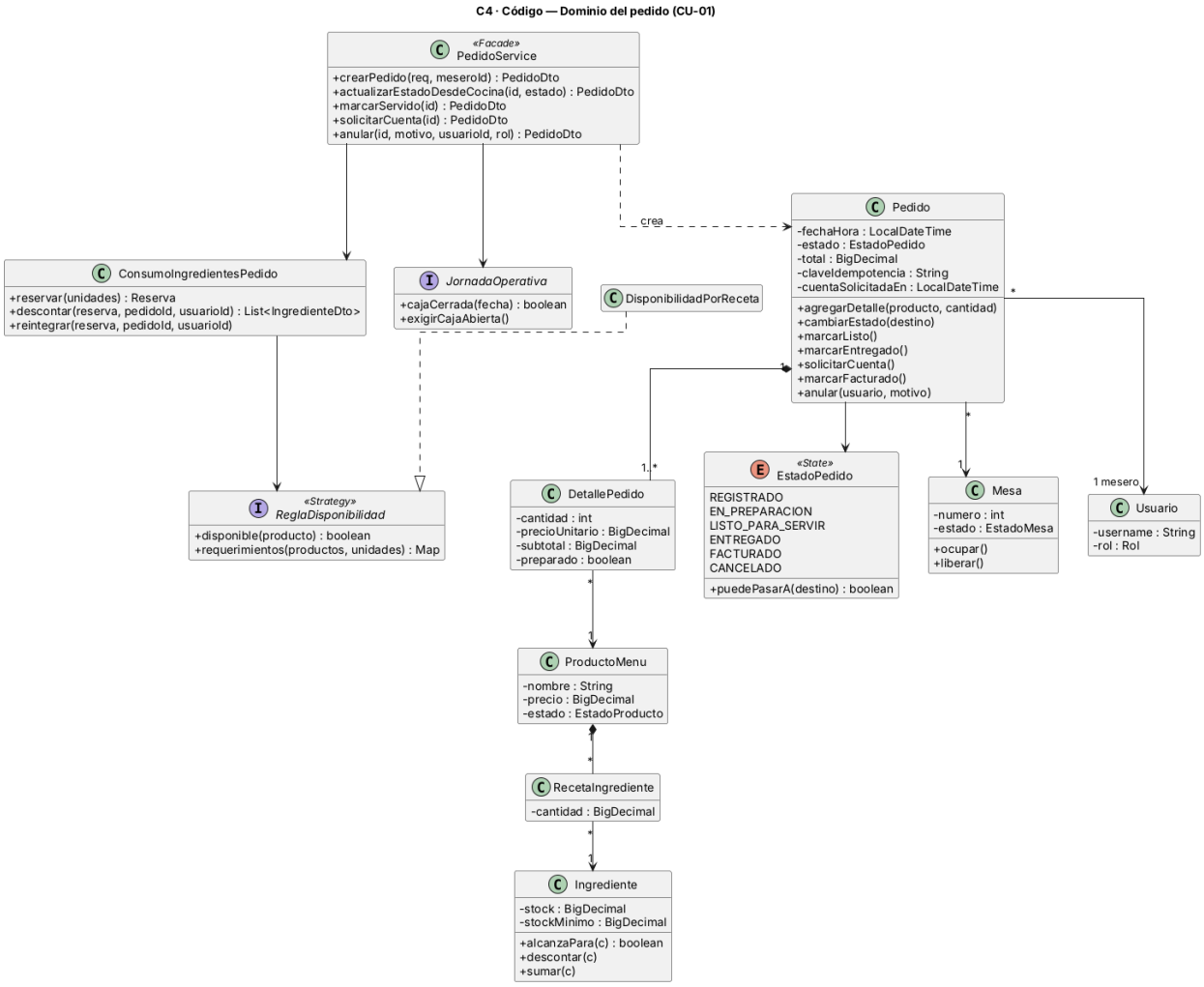


Figura 6. *C4 · Código: dominio del pedido*

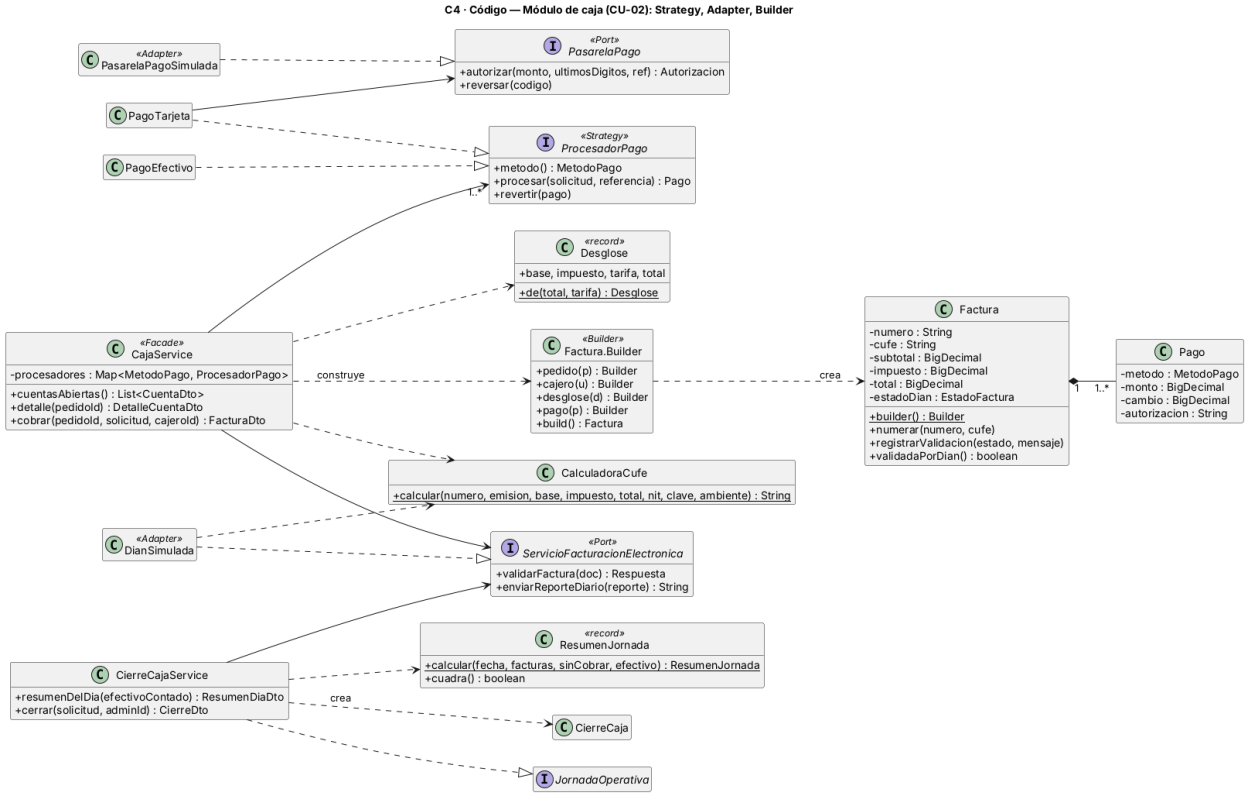


Figura 7. *C4 · Código: módulo de caja*

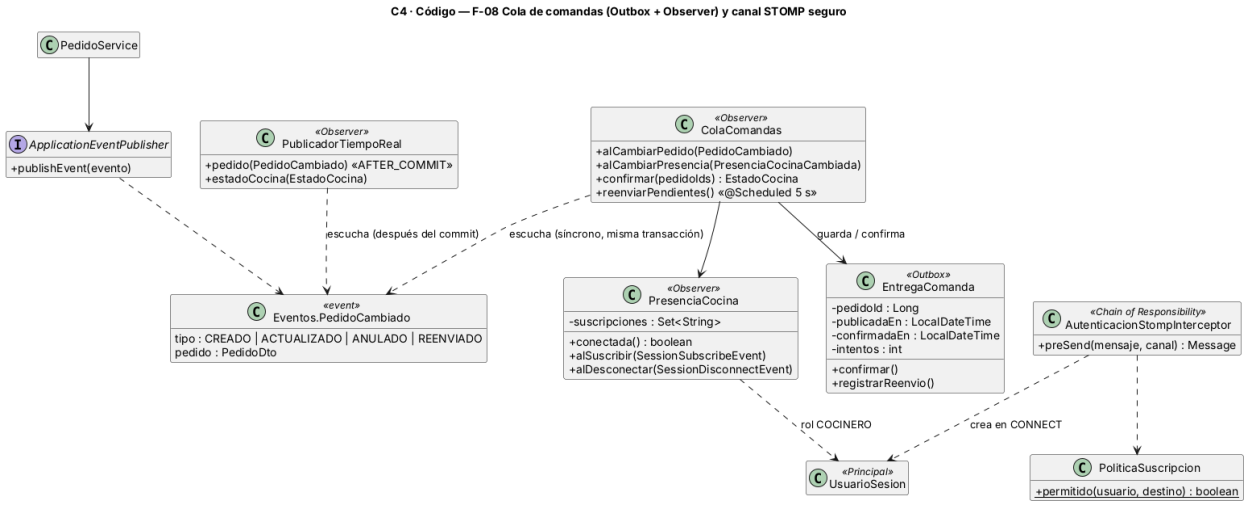


Figura 8. *C4 · Código: cola de comandas y canal STOMP seguro*

Modelo de datos

Tabla 9. *Tablas de la base de datos*

Tabla	Contenido	Migración
usuario, mesa	Usuarios con rol y hash BCrypt; mesas con estado	V1
ingrediente, producto_menu, receta_ingrediente	Inventario y catálogo de recetas (cantidad por unidad de plato)	V1

Tabla	Contenido	Migración
pedido, detalle_pedido	Pedido con estado, total, auditoría de anulación, clave de idempotencia y cuenta solicitada	V1, V3
movimiento_inventario	Entradas, salidas por pedido y reintegros	V1
entrega_comanda	Outbox de comandas: publicada, confirmada (ACK), intentos	V3
factura, pago	Factura con CUFE y estado DIAN; uno o varios pagos por factura	V3
cierre_caja	Un cierre por fecha con totales, inconsistencias aceptadas y comprobante	V3

5. Interacciones y flujos de datos

5.1 Flujos de datos entre componentes

Existen dos canales de comunicación entre la SPA y el backend. Las **órdenes** (crear un pedido, cambiar su estado, cobrar, cerrar caja) viajan por **REST/HTTPS** con el JWT en la cabecera `Authorization`; cada operación es una transacción que valida reglas de negocio y responde con un DTO o con un error de negocio con código estable (por ejemplo `INGREDIENTES_INSUFICIENTES`, `PAGO_RECHAZADO`, `CAJA_CERRADA`). Los **cambios de estado** se difunden por **WebSocket/STOMP** solo después del commit (`@TransactionalEventListener(AFTER_COMMIT)`), para que ninguna pantalla muestre un cambio que luego se revierta.

Tabla 10. *Tópicos de eventos en tiempo real*

Tópico STOMP	Publica	Suscriptores (rol)	Contenido
/topic/mesas	Pedido, caja	Mesero, cajero, administrador	Estado de una mesa y su pedido activo
/topic/menu	Pedido, inventario	Mesero, administrador	Señal para recalcular disponibilidad
/topic/cocina	Pedido, cola de comandas	Cocinero, administrador	Comanda creada, actualizada, anulada o reenviada
/topic/estado-cocina	Cola de comandas	Mesero, cocinero, administrador	Cocina conectada y comandas sin ACK
/topic/meseros/{id}	Pedido, recordatorio	Solo el mesero {id}	Plato listo, recordatorio o atendido
/topic/inventario/alertas	Pedido	Administrador	Ingredientes en nivel mínimo
/topic/caja	Pedido, caja	Cajero, administrador	Cambió la lista de cuentas o se cerró la caja

Cuando una conexión se restablece, el cliente STOMP vuelve a suscribirse y cada pantalla recarga su estado por REST; así ningún evento perdido durante la desconexión deja la vista desactualizada. Para la cocina, además, el servidor mantiene la cola persistente de comandas (F-08).

5.2 Vista C4+1: diagramas de secuencia

La vista "+1" (escenarios, heredada del modelo 4+1 de Kruchten) muestra el comportamiento en tiempo de ejecución de los casos de uso críticos a través de los contenedores y componentes. Se modelan seis escenarios.

Secuencia 1 · Registrar pedido (Contrato 1)

Incluye la idempotencia del reintento de la tablet, la revalidación de stock con bloqueo de filas (SWR-05), el descuento en la misma transacción (SWR-06), el registro en la cola de comandas y el ACK de la cocina.

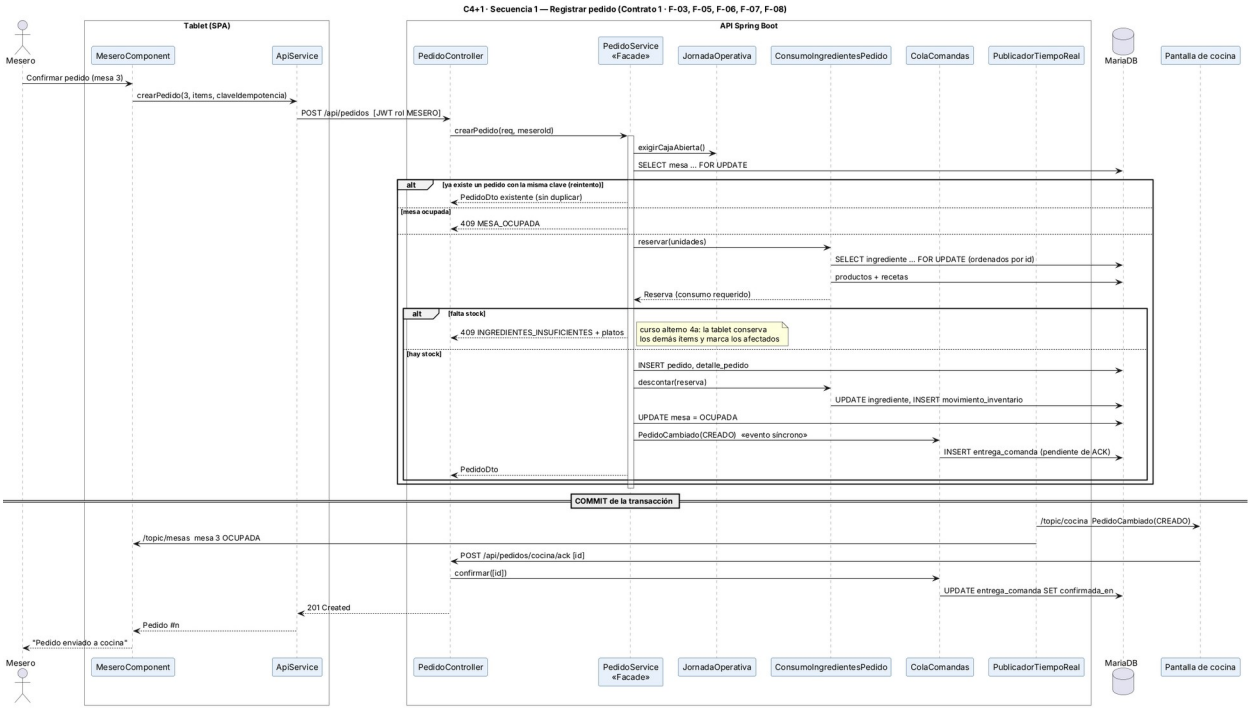


Figura 9. *Secuencia 1 · Registrar pedido*

Secuencia 2 · Preparar y notificar al mesero (Contrato 2)

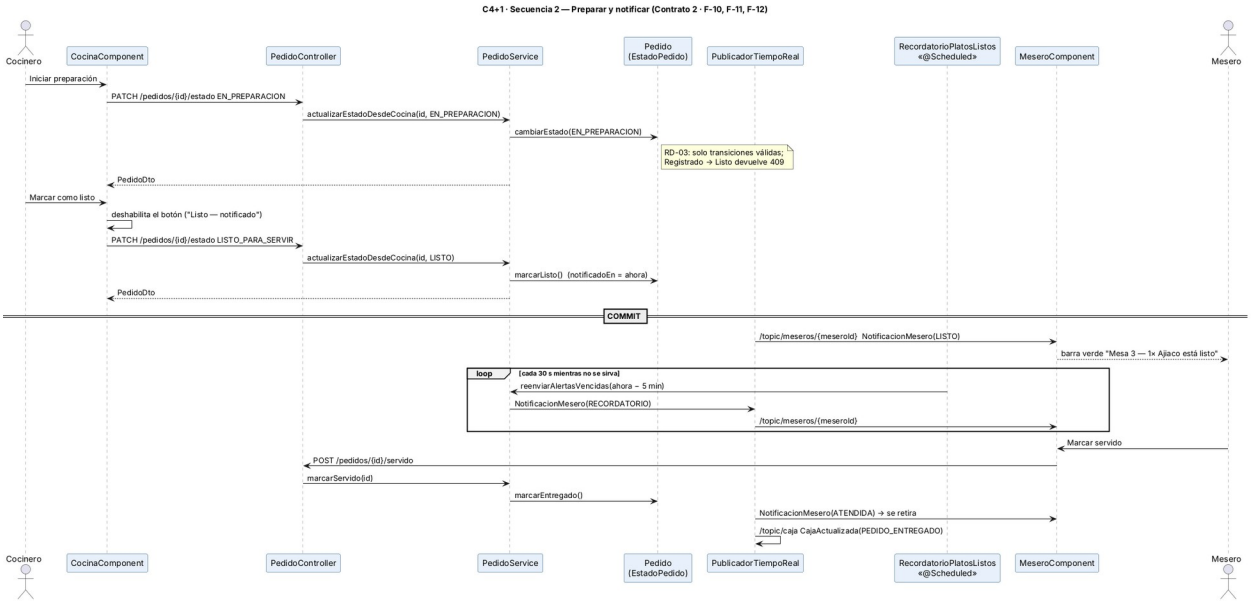


Figura 10. *Secuencia 2 · Preparación, notificación y entrega*

Secuencia 3 · Cobrar y generar la factura electrónica (Contrato 3)

Muestra la estrategia de pago por método, la pasarela como adaptador, la construcción de la factura con el Builder, el cálculo del CUFE, la validación ante la DIAN y la compensación (reverso de la tarjeta) si la DIAN rechaza.

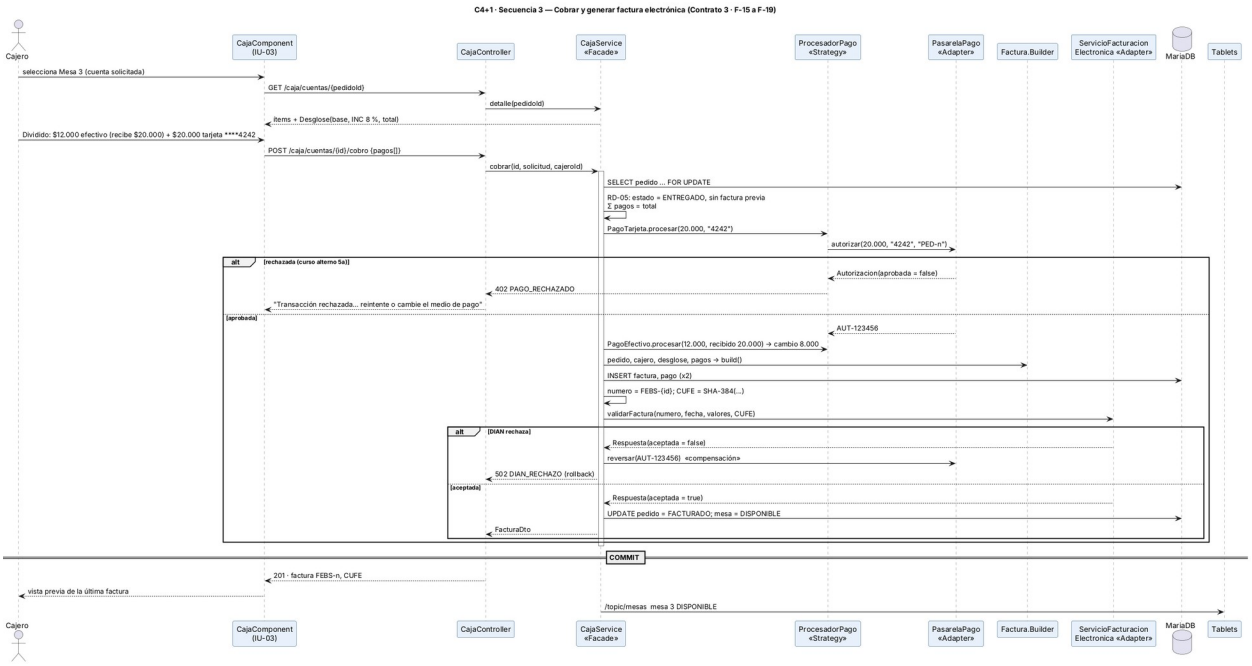


Figura 11. *Secuencia 3 · Cobro y factura electrónica*

Secuencia 4 · Pérdida de conexión y reenvío de comandas (F-08)

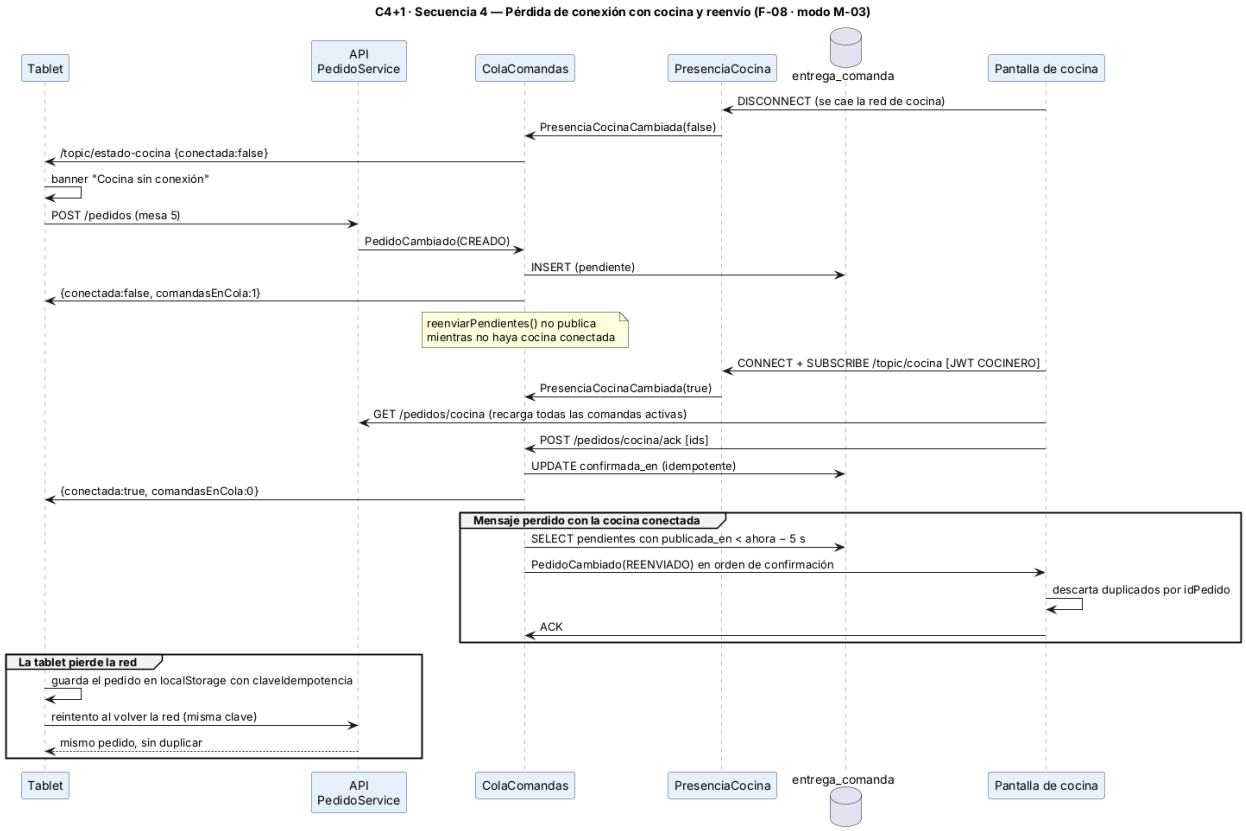


Figura 12. *Secuencia 4 · Cola de comandas, ACK y reenvío*

Secuencia 5 · Conciliación y cierre de caja

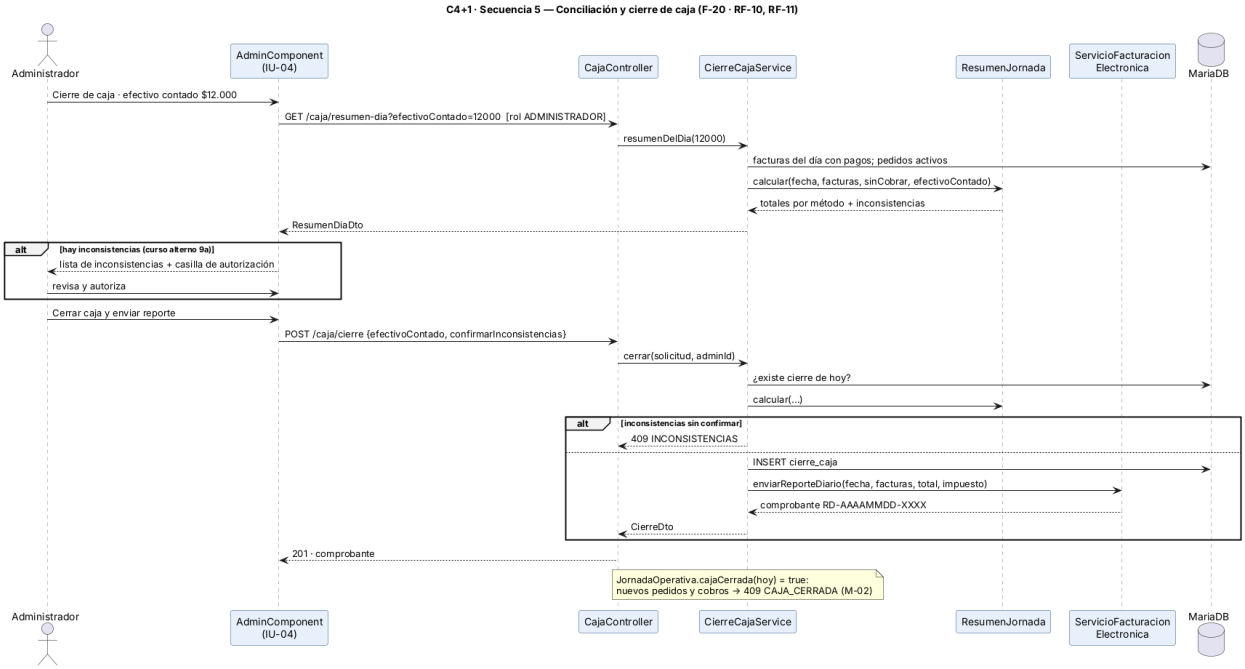


Figura 13. *Secuencia 5 · Cierre de caja*

Secuencia 6 · Autenticación REST y WebSocket

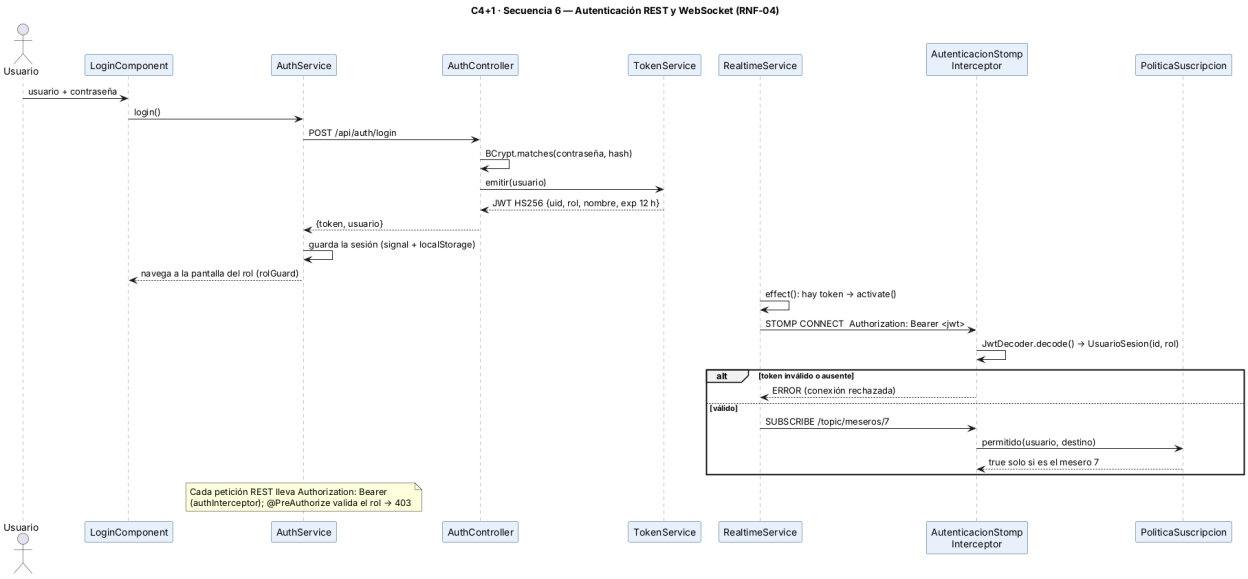


Figura 14. *Secuencia 6 · Autenticación y autorización*

6. Decisiones de diseño y justificaciones

6.1 Registro de decisiones de arquitectura (ADR)

Tabla 11. *Decisiones de arquitectura*

ID	Decisión	Alternativas consideradas	Justificación y consecuencias
AD-01	Monolito modular en lugar de microservicios	Microservicios por módulo	Un solo restaurante, equipo pequeño y un único servidor: menor costo operativo y transacciones locales. Los módulos con dependencias explícitas permiten extraer un servicio después.
AD-02	WebSocket/STOMP para tiempo real	Consultas periódicas (polling); Server-Sent Events	Cumple RNF-02 (≤ 2 s) sin sobrecargar la red; STOMP aporta tópicos y suscripciones. Broker en memoria suficiente para un solo nodo.
AD-03	Publicar eventos después del commit	Publicar dentro de la transacción	Evita mostrar cambios que se revierten. Si el servidor cae justo tras el commit el evento se pierde; lo cubre la recarga al reconectar y la cola F-08.
AD-04	Bloqueo pesimista (SELECT … FOR UPDATE) de mesa e ingredientes, siempre en el mismo orden	Bloqueo optimista con reintentos	Garantiza que dos tablets no consuman el mismo stock (SWR-05) sin reintentos en el cliente; el orden fijo evita interbloqueos. Costo: contención breve en horas pico.
AD-05	Outbox con ACK para F-08 (servidor) + cola local con idempotencia (tablet)	Solo recargar al reconectar; colas externas (RabbitMQ)	Resuelve el conflicto RF-02/M-03 sin agregar infraestructura; reenvío ordenado y sin duplicados.
AD-06	Puertos y adaptadores para la pasarela y la DIAN, con adaptadores simulados	Integrar directamente el SDK del banco y el webservice	Aísla el dominio de los cambios externos, permite pruebas sin red y desarrollar antes de tener credenciales reales.
AD-07	Strategy para métodos de pago y para la regla de disponibilidad	switch por método; cálculo fijo en el servicio	Abierto/cerrado: agregar transferencia o un margen de seguridad de stock es agregar una clase.
AD-08	Impuesto al consumo 8 % incluido en el precio	IVA 19 % sumado al precio	Es el impuesto que liquidan los restaurantes; el total coincide con la suma del menú (RF-06). Tarifa configurable.
AD-09	JWT sin estado + RBAC con @PreAuthorize; JWT también en STOMP CONNECT	Sesiones en servidor; WebSocket abierto	Escala sin estado compartido; cierra el pendiente de seguridad del canal en tiempo real (RNF-04).
AD-10	Migraciones versionadas con Flyway; Hibernate no modifica el esquema	ddl-auto=update	Esquema reproducible y auditable en todos los entornos.
AD-11	Compensación (reverso) en lugar de transacción distribuida en el cobro	Two-phase commit	Los servicios externos no participan en la transacción local; si la DIAN rechaza se reversa la tarjeta y se deshace la factura.

6.2 Consideraciones de calidad

Rendimiento

- Consultas con grafos de entidades (`@EntityGraph`) y `join fetch` para evitar el problema N+1 al cargar pedidos con detalles o productos con receta.
- Solo se bloquean los ingredientes que usa el pedido; la lectura del menú no bloquea.
- Eventos pequeños por WebSocket; la SPA actualiza solo el elemento afectado (por ejemplo, una mesa).

Escalabilidad

- API sin estado (JWT): se pueden levantar varias instancias detrás de Nginx. Para ello el broker en memoria debe reemplazarse por un broker externo (RabbitMQ con STOMP) y la presencia de cocina debe guardarse en un almacén compartido.
- El restaurante objetivo tiene 12 mesas y 4 tipos de dispositivo; un solo nodo cubre holgadamente la carga.

Mantenibilidad

- Paquetes por dominio, servicios con una sola responsabilidad y nombres del lenguaje del negocio.
- Reglas puras sin base de datos (`CalculadoraReceta`, `Desglose`, `ResumenJornada`, `CalculadoraCufe`, `PoliticaSuscripcion`, `EstadoPedido`) fáciles de probar.
- API documentada en Swagger, diagramas como código (PlantUML) versionados junto al código, y CI que impide integrar código que no compila o no pasa las pruebas.

Fiabilidad

- Transacciones atómicas para pedido + inventario + cola, y para factura + pagos + estado del pedido + mesa.
- Cola de comandas persistente con ACK y reenvío; cola local en la tablet con clave de idempotencia.
- Restricciones en la base de datos (únicas y CHECK) como última línea de defensa: una factura por pedido, un cierre por fecha, stock no negativo.

7. Patrones de diseño utilizados

7.1 Patrones GoF y empresariales

Tabla 12. *Patrones de diseño aplicados*

Patrón	Categoría	Dónde se aplica	Problema que resuelve
Facade	Estructural	PedidoService, CajaService	Los controladores llaman una operación del sistema; el servicio coordina repositorios, inventario, pasarela, DIAN y eventos.
Strategy	Comportamiento	ProcesadorPago (PagoEfectivo, PagoTarjeta); ReglaDisponibilidad (DisponibilidadPorReceta)	Algoritmos intercambiables seleccionados en tiempo de ejecución sin condicionales.
Adapter	Estructural	PasarelaPago → PasarelaPagoSimulada; ServicioFacturacionElectronica → DianSimulada; PublicadorTiempoReal	Adaptar interfaces externas (banco, DIAN, broker STOMP) a la interfaz que espera el dominio.
Observer	Comportamiento	ApplicationEventPublisher + Eventos + PublicadorTiempoReal, ColaComandas, PresenciaCocina	Notificar a varios interesados (pantallas, cola de comandas) cuando cambia un pedido, sin acoplarlos.
State (declarativo)	Comportamiento	EstadoPedido con tabla de transiciones	El comportamiento permitido depende del estado (RD-03).
Builder	Creacional	Factura.builder()	Construir un objeto complejo solo cuando tiene todos los datos obligatorios.
Chain of Responsibility	Comportamiento	AutenticacionStompInterceptor en la cadena de ChannelInterceptor; filtros de Spring Security	Cada eslabón autentica o autoriza el mensaje antes de que llegue al broker.
Singleton (gestionado)	Creacional	Beans de Spring (@Service, @Component)	Una instancia compartida por servicio, creada e inyectada por el contenedor.
Repository	Empresarial (DDD)	Interfaces *Repository de Spring Data	Abstraer la persistencia del dominio.
DTO	Empresarial	PedidoDto, CajaDtos, Eventos	No exponer entidades JPA por la API ni por WebSocket.
Transactional Outbox	Integración	EntregaComanda / ColaComandas	Entrega confiable de mensajes ligada a la transacción de negocio.

7.2 Ejemplos de aplicación en el código

Strategy: procesadores de pago

```
public interface ProcesadorPago {
    MetodoPago metodo();
    Pago procesar(SolicitudPago solicitud, String referencia);
    default void revertir(Pago pago) { }
}

// CajaService recibe todas las estrategias y las indexa por método:
```

```
this.procesadores = new EnumMap<>(procesadores.stream()
        .collect(Collectors.toMap(ProcesadorPago::metodo, Function.identity())));
...
procesados.add(procesador(parte.metodo()).procesar(parte, referencia));
```

Adapter (puerto y adaptador): pasarela de pago

```
public interface PasarelaPago {                        // puerto (dominio)
    Autorizacion autorizar(BigDecimal monto, String ultimosDigitos, String referencia);
    void reversar(String codigoAutorizacion);
}

@Component
public class PasarelaPagoSimulada implements PasarelaPago { ... }  // adaptador
```

Builder: factura electrónica

```
Factura.Builder builder =
Factura.builder().pedido(pedido).cajero(cajero).desglose(desglose);
pagos.forEach(builder::pago);
Factura factura = facturas.save(builder.build());   // build() valida que esté completa
```

Observer: eventos de dominio publicados después del commit

```
eventos.publishEvent(new Eventos.PedidoCambiado(TipoCambioPedido.CREADO, dto));

@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT, fallbackExecution =
true)
public void pedido(Eventos.PedidoCambiado evento) {
    broker.convertAndSend("/topic/cocina", evento);
}
```

State: máquina de estados del pedido

```
private static final Map<EstadoPedido, Set<EstadoPedido>> TRANSICIONES = Map.of(
        REGISTRADO, EnumSet.of(EN_PREPARACION, CANCELADO),
        EN_PREPARACION, EnumSet.of(LISTO_PARA_SERVIR),
        LISTO_PARA_SERVIR, EnumSet.of(ENTREGADO),
        ENTREGADO, EnumSet.of(FACTURADO), ...);

public boolean puedePasarA(EstadoPedido destino) { return
TRANSICIONES.get(this).contains(destino); }
```

7.3 Principios SOLID

Tabla 13. *Aplicación de SOLID*

Principio	Aplicación
S · Responsabilidad única	PedidoService ya no calcula ni descuenta inventario (ConsumoIngredientesPedido) ni lista mesas (MesaService); CajaService cobra, CierreCajaService concilia y cierra.
O · Abierto/cerrado	Nuevos métodos de pago o reglas de disponibilidad se agregan con nuevas clases que implementan ProcesadorPago o ReglaDisponibilidad.
L · Sustitución de Liskov	PasarelaPagoSimulada y un futuro adaptador bancario son intercambiables; las pruebas usan dobles de prueba de las mismas interfaces.
I · Segregación de interfaces	Interfaces pequeñas y específicas: PasarelaPago (2 operaciones), JornadaOperativa (1), ReglaDisponibilidad (2).

Principio	Aplicación
D · Inversión de dependencias	PedidoService depende de JornadaOperativa, no de CierreCajaService; CajaService depende de puertos, no de los adaptadores concretos.

7.4 Refactorizaciones aplicadas al CU-01 (código limpio)

Tabla 14. *Catálogo de refactorizaciones*

Mal olor (code smell)	Técnica de refactorización	Resultado
Long Method / Large Class: PedidoService.crearPedido tenía unas 90 líneas y ~350 la clase	Extract Method + Extract Class	crearPedido de ~25 líneas con métodos auxiliares de una sola tarea; nueva clase ConsumoIngredientesPedido
Duplicate Code: bloqueo y cálculo de ingredientes repetido en crearPedido y anular	Extract Class (DRY)	reservar / descontar / reintegrar en un solo lugar
Feature Envy: MesaController dependía de PedidoService	Move Method	MesaService con listar() y pedidoActivo()
Switch Statements (potencial) por método de pago	Replace Conditional with Polymorphism / Strategy	ProcesadorPago
Inappropriate Intimacy: Mesa.setEstado usado desde afuera	Encapsulate Field / métodos con intención	ocupar(), liberar(), estaOcupada()
Condicionales anidados en validaciones	Introduce Guard Clauses	validarProductos, validarStock, validarQuePuedeAnular, bloquearPedidoCobrable
Dependencia circular de módulos pedido ↔ caja	Dependency Inversion (interfaz)	JornadaOperativa en common

Se respetaron las reglas de código limpio del curso: nombres de clases como sustantivos del dominio (`Factura`, `Desglose`, `ColaComandas`), métodos como verbos que expresan intención (`calcular`, `reservar`, `exigirCajaAbierta`), constantes en MAYÚSCULAS_CON_GUION_BAJO (`TOPICO_COCINA`, `CONSUMIDOR_FINAL`), funciones cortas sin efectos secundarios ocultos y la regla del boy scout en los archivos que se tocaron.

8. Gestión de la calidad y pruebas

8.1 Estrategia de prueba

Se sigue la **pirámide de pruebas**: muchas pruebas unitarias rápidas sobre la lógica de dominio pura, algunas pruebas de servicio con dobles de prueba (Mockito) y una prueba de humo de extremo a extremo contra el sistema completo levantado con Docker Compose y una base de datos real.

Tabla 15. *Niveles de prueba*

Nivel	Herramienta	Qué cubre
Unitario (dominio)	JUnit 5 + AssertJ	Máquina de estados, receta, desglose, CUFE, conciliación, política de suscripción, estrategias de pago
Servicio (con dobles)	JUnit 5 + Mockito	Contrato 3 completo (factura, mesa liberada, reverso de tarjeta), cola de comandas
Extremo a extremo (API)	scripts/prueba-humo.sh (curl + jq) contra Docker Compose	Las seis funcionalidades de la demostración y sus cursos alternos, con MariaDB real
Interfaz	Guion de demostración manual	IU-01 a IU-04 contra los mockups
Estática / compilación	Compilador Java, TypeScript estricto, plantillas Angular estrictas	Errores de tipos en código y plantillas

8.2 Pruebas automatizadas y trazabilidad

Tabla 16. *Pruebas automatizadas*

Suite	Requisitos verificados	Casos
EstadoPedidoTest	F-10, F-13, RD-03	Secuencia válida, saltos, retrocesos, anulación
CalculadoraRecetaTest	F-02, F-06	Disponibilidad por ingrediente; consumo multiplicado y sumado
PoliticaSuscripcionTest	RNF-04, F-11	Mesero solo su tópico; cocina y caja; tópicos desconocidos
ColaComandasTest	F-08	Registro pendiente, ACK idempotente, sin reenvío con cocina desconectada
DesgloseTest	F-15, RF-06	Base + INC = total; tarifa 0
CalculadoraCufeTest	F-19, RF-07	96 hexadecimales; determinista; cambia con los datos
ProcesadoresPagoTest	F-16, F-17	Cambio; efectivo insuficiente; tarjeta aprobada y rechazada
CajaServiceTest	F-18, F-19, Contrato 3	Pago dividido, pagos que no cuadran, DIAN rechaza → reverso, pedido no entregado
ResumenJornadaTest	F-20, RF-10, RF-11	Totales por método; factura sin CUFE, pedidos abiertos, descuadre de efectivo
prueba-humo.sh	F-01 a F-20, RNF-04, M-02	25 verificaciones por la API con base de datos real

8.3 Integración continua y métricas

El flujo de GitHub Actions (`.github/workflows/ci.yml`) se ejecuta en cada *push* y *pull request* con tres trabajos: (1) **backend**: `mvn verify` con pruebas y reporte de cobertura JaCoCo publicado como artefacto; (2) **frontend**: `ng build --configuration production` con plantillas estrictas; (3) **extremo a extremo**: `docker compose up --build` y la prueba de humo. Una rama solo se integra si los tres trabajos pasan.

Tabla 17. *Métricas de calidad del código*

Métrica	Valor (v0.3)
Clases Java de producción	83 archivos · 4.601 líneas
Pruebas Java	10 clases · 26 casos de prueba
Frontend	1.732 líneas de TypeScript · 577 líneas de plantillas
Verificaciones de la prueba de humo	25
Cobertura de líneas	Reporte JaCoCo generado en cada ejecución de CI (artefacto "cobertura-jacoco")
Funcionalidades con prueba automatizada	18 de 21 (F-04, F-09 y F-21 se verifican en la interfaz)

Durante el desarrollo de esta versión el código Java nuevo se compiló contra las interfaces de las bibliotecas y la lógica de dominio pura (desglose, CUFE, conciliación, política de suscripción, estrategias de pago, máquina de estados) se ejecutó de forma aislada con resultados correctos; el TypeScript se verificó con el compilador en modo estricto. La ejecución completa (Maven, Angular y prueba de humo con MariaDB) queda registrada en la pestaña *Actions* del repositorio.

9. Despliegue y configuración

9.1 Entorno de implementación

El sistema se despliega con Docker Compose en un servidor del restaurante (o en la nube) conectado a la red Wi-Fi local. Los dispositivos de los usuarios solo necesitan un navegador.

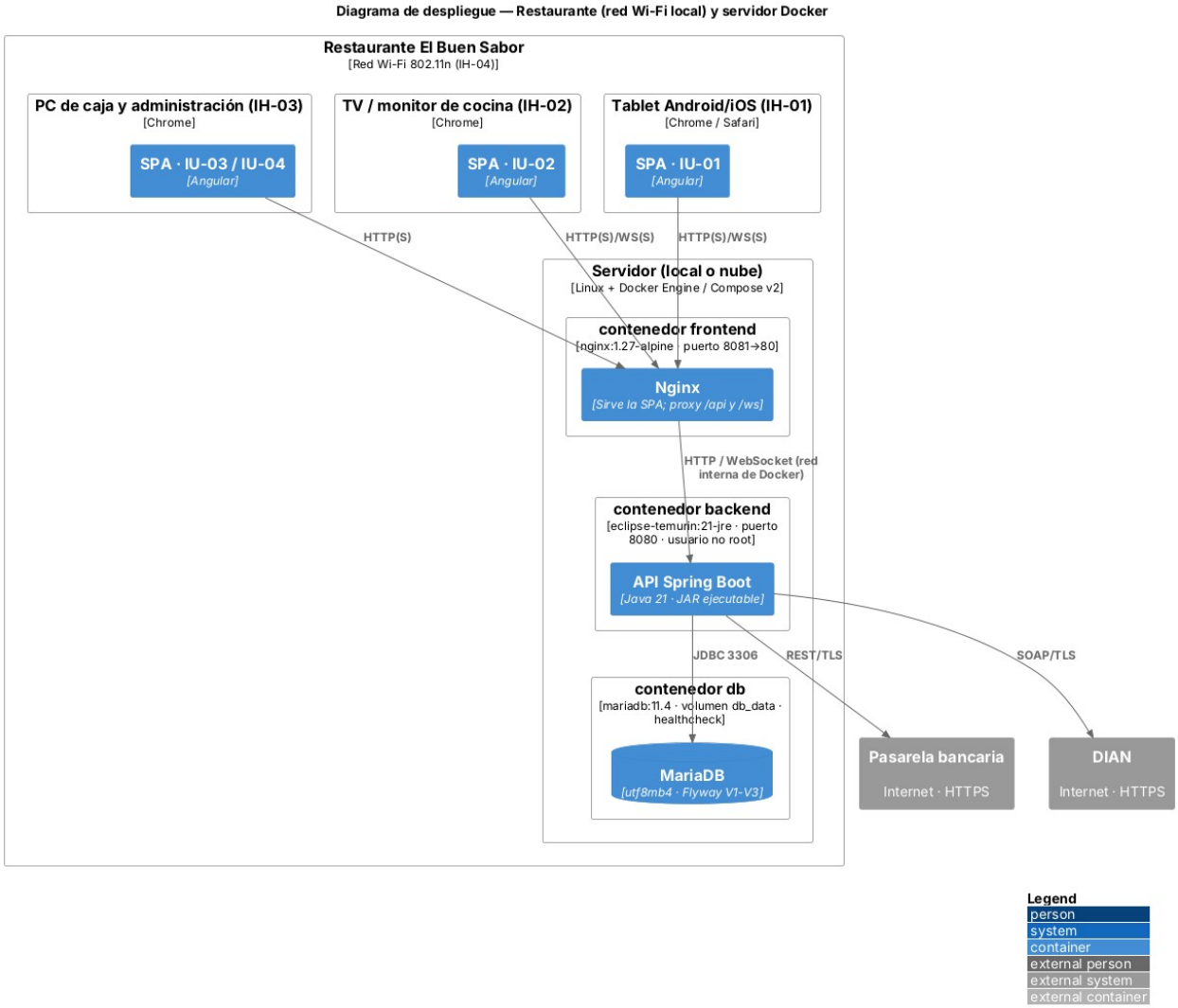


Figura 15. *Diagrama de despliegue*

Tabla 18. *Contenedores*

Contenedor	Imagen	Puerto	Detalles
db	mariadb:11.4	3307 → 3306	utf8mb4; volumen db_data; healthcheck; reinicio automático
backend	Construida: maven:3.9-temurin-21 → eclipse-temurin:21-jre	8080	JAR ejecutable, usuario no root, MaxRAMPercentage=75; espera a que la base de datos esté sana
frontend	Construida: node:22-alpine →	8081 → 80	SPA compilada para producción; proxy

Contenedor	Imagen	Puerto	Detalles
	nginx:1.27-alpine		/api, /ws y Swagger; caché de estáticos

9.2 Configuración

Tabla 19. *Variables de configuración*

Variable (.env)	Uso	Valor por defecto
RESTAURANTE_NOMBRE	Nombre en el encabezado de las pantallas	El Buen Sabor
DB_NAME, DB_USER, DB_PASSWORD, DB_ROOT_PASSWORD	Credenciales de MariaDB	Valores de desarrollo
JWT_SECRET	Clave HMAC de los tokens (≥ 32 caracteres)	Debe cambiarse en producción
CORS_ORIGENES	Orígenes permitidos (incluye la red local de las tablets)	localhost y 192.168.*, 10.*
MINUTOS_REENVIO_ALERTA	Recordatorio de plato listo (curso alterno 6a)	5
SEGUNDOS_REENVIO_COMANDA	Reenvío de comandas sin ACK (F-08)	5
FACTURA_NIT, FACTURA_PREFIJO, FACTURA_TARIFA_IMPUESTO	Datos de facturación electrónica	900123456 · FEBS · 0,08
FRONTEND_PUERTO, BACKEND_PUERTO, DB_PUERTO_LOCAL, TZ	Puertos y zona horaria	8081 · 8080 · 3307 · America/Bogota

9.3 Proceso de despliegue y actualización

1. Clonar el repositorio y copiar `.env.example` a `.env`; ajustar `JWT_SECRET` y las contraseñas.
2. Ejecutar `docker compose up -d --build`. Flyway crea o actualiza el esquema (V1, V2, V3) y aplica la semilla de stock.
3. Verificar `http://<servidor>:8080/actuator/health` y ejecutar `./scripts/prueba-humo.sh` sobre una base nueva.
4. Abrir `http://<IP-del-servidor>:8081` en cada tablet, TV de cocina y PC de caja.

Actualización: `git pull` y `docker compose up -d --build`. Las migraciones nuevas se aplican solas al arrancar el backend; nunca se editan migraciones ya aplicadas. Se recomienda actualizar fuera del horario de atención (modo M-04 Mantenimiento) y respaldar el volumen `db_data` antes (`mysqldump`). **Producción:** publicar Nginx con certificado TLS (RNF-01) y reemplazar los adaptadores simulados por los reales de la entidad bancaria y la DIAN.

10. Consideraciones de seguridad

10.1 Medidas implementadas

- **Autenticación:** usuario y contraseña con hash BCrypt; el servidor emite un JWT firmado con HMAC-SHA256 que expira en 12 horas y lleva el id, el nombre y el rol.
- **API sin estado:** CSRF deshabilitado porque no se usan cookies de sesión; CORS restringido a los orígenes configurados.
- **Autorización por operación:** `@PreAuthorize` en cada endpoint; un rol no autorizado recibe 403 (verificado en la prueba de humo).
- **Canal en tiempo real:** la trama STOMP CONNECT exige el JWT; cada SUBSCRIBE se valida contra `PoliticaSuscripcion`, de modo que un mesero solo recibe sus propias notificaciones.
- **Reglas de propiedad:** solo el mesero que tomó un pedido (o un administrador) puede anularlo.
- **Datos de pago:** el sistema nunca recibe ni guarda el número completo de la tarjeta (solo los últimos 4 dígitos y el código de autorización).
- **Validación de entrada:** Bean Validation en todas las solicitudes y errores con códigos estables, sin trazas internas hacia el cliente.
- **Auditoría:** usuario y fecha en anulaciones, movimientos de inventario, pagos, facturas y cierres.
- **Contenedores:** el backend se ejecuta con un usuario sin privilegios; los secretos se inyectan por variables de entorno y `.env` no se versiona.
- **Pendiente para producción:** TLS en Nginx (RNF-01), rotación de `JWT_SECRET`, límites de intentos de inicio de sesión y certificado digital para la firma de facturas.

10.2 Gestión de roles y permisos

Tabla 20. *Matriz de roles y permisos (RBAC)*

Operación	Mesero	Cocinero	Cajero	Admin
Ver mesas y pedido activo	✓		✓	✓
Ver menú con disponibilidad	✓			✓
Registrar pedido / servir / solicitar cuenta	✓			
Anular pedido	✓ (propio)			✓
Ver comandas, cambiar estado, ACK		✓		Ver
Ver cuentas, cobrar y facturar			✓	✓
Registro de caja del día (facturas)			✓	✓
Resumen del día y cierre de caja				✓
Inventario, entradas y recetas				✓

11. Referencias y recursos adicionales

1. Repositorio del proyecto: https://github.com/SSeanjjo/gestion-restaurantes (rama feature/parcial-2-arquitectura-cu02).
2. Brown, S. The C4 model for visualising software architecture. https://c4model.com
3. Kruchten, P. (1995). Architectural Blueprints — The "4+1" View Model of Software Architecture. IEEE Software, 12(6).
4. Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
5. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
6. Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2.ª ed.). Addison-Wesley.
7. ISO/IEC/IEEE 29148:2018. Systems and software engineering — Requirements engineering.
8. ISO/IEC 25010:2023. Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.
9. Richardson, C. Pattern: Transactional outbox. https://microservices.io/patterns/data/transactional-outbox.html
10. DIAN. Resolución 000042 de 2020 y Anexo Técnico de Factura Electrónica de Venta.
11. Documentación de Spring Boot, Spring Security y Spring WebSocket (STOMP): https://docs.spring.io
12. Documentación de Angular: https://angular.dev · C4-PlantUML: https://github.com/plantuml-stdlib/C4-PlantUML
13. Student Book del proyecto (Secciones 1–4) y Plan de Requisitos 2026 V3 (CU-01 y CU-02).